

SynapseQuill — Arquitectura

Cómo funciona el sistema, de principio a fin

Junio 2026

Introducción

SynapseQuill es un sistema que convierte datos deportivos (y otros temas) en contenido listo para publicar: vídeos narrados de partidos de fútbol, textos para redes sociales, explicaciones científicas y resúmenes de mercado. Todo se apoya en inteligencia artificial, pero cada pieza tiene un trabajo concreto y acotado.

Este documento explica **todos los conceptos y procesos** del sistema. Está pensado para leerse de principio a fin sin conocimientos previos: primero los conceptos básicos, luego cada flujo paso a paso, con ejemplos cuando algo es difícil de visualizar. Las **cajas técnicas** (“🔧 Detalle técnico”) amplían para quien quiera el porqué exacto; se pueden saltar sin perder el hilo.

Parte 1 · Conceptos básicos

Antes de ver los procesos conviene entender el vocabulario. Estos términos aparecen una y otra vez.

¿Qué es un LLM?

Un **LLM** (Large Language Model, “modelo de lenguaje grande”) es una IA que **lee texto y escribe texto**. Le das unas instrucciones (el *prompt*) y te devuelve una respuesta redactada.

Ejemplo. Si al LLM le das: “Eres un relator de fútbol. Datos: Valencia 2, Barcelona 1. Goles: López (min 12), Pérez (min 70). Escribe una narración emocionante de 100 palabras”, te devuelve un texto como: “¡Qué partido en Mestalla! López abrió la lata muy pronto...”.

En SynapseQuill los LLM escriben **las narraciones, los títulos de YouTube y los textos de redes**. Los proveedores usados (Cerebras, Groq, Gemini) son intercambiables: todos hacen lo mismo, solo cambia quién ejecuta el modelo.

¿Qué es un embedding?

Un **embedding** es lo contrario de un LLM en cuanto a salida: coge un texto y lo convierte en una **lista de números** que captura su *significado*. Textos parecidos producen números parecidos.

Ejemplo. “perro” y “cachorro” tendrán números muy cercanos entre sí, y muy lejanos de “hipoteca”. Así un ordenador puede medir *cuánto se parecen* dos frases aunque no compartan ninguna palabra.

Esto sirve para **buscar por significado** (no por palabra exacta). Es la base del “RAG” que veremos en el Laboratorio de Ciencia. El modelo de embeddings que usa el proyecto, `bge-small`, es de **BAAI** (un instituto de investigación de Pekín), es open source y corre **en tu propio ordenador**, gratis.

¿Qué es una API?

Una **API** es una dirección web que, en vez de devolver una página para humanos, devuelve **datos** para programas. SynapseQuill llama a APIs para obtener resultados de partidos (ESPN), generar imágenes (FLUX), convertir texto en voz (Edge-TTS, ElevenLabs), etc.

Ejemplo. El programa pide a la API de ESPN “dame el partido 748519” y ESPN responde con un bloque de datos: equipos, marcador, goleadores y minutos.

¿Qué es TTS?

TTS = *Text To Speech*, “texto a voz”. Convierte un guion escrito en un archivo de audio con una voz hablando. El narrador por defecto es **Jorge** (Edge-TTS, voz mexicana), gratis e ilimitado.

¿Qué es un guardrail?

Un **guardrail** (“barrera de protección”) es una comprobación de seguridad que verifica que la IA **no se ha inventado nada**. En los vídeos, antes de dar por buena una narración, se comprueba que el marcador y los nombres coincidan con los datos reales del partido.

¿Qué es RAG?

RAG = *Retrieval-Augmented Generation*, “generación aumentada por recuperación”. En lugar de dejar que la IA responda “de memoria” (donde puede inventar), primero se **recuperan** documentos reales relevantes y se le pide que responda **basándose solo en ellos**. Resultado: respuestas fundamentadas y verificables.

¿Qué es un agente / LangGraph?

Un **agente** es un mini-asistente de IA con un rol concreto (p. ej. “experto en finanzas”) que puede usar **herramientas** (llamar a una API) por su cuenta antes de responder. **LangGraph** es la tecnología que permite coordinar varios agentes con un **supervisor** que decide quién atiende cada petición.

Parte 2 · Visión general de la arquitectura

El sistema tiene tres capas:

1. **Frontend (la web)** — lo que ves: páginas de Dashboard, Partidos, Crear, Laboratorio, Biblioteca, Arquitectura y Ajustes. Hecho en React.
2. **Backend (la API)** — recibe las peticiones de la web y orquesta el trabajo. Hecho en Python con FastAPI. Aquí viven los “flujos”.
3. **Servicios externos** — las APIs e IAs que hacen el trabajo pesado (escribir, generar imágenes, poner voz, dar datos).

Diseño intercambiable: para cada tarea (texto, imagen, voz, datos de fútbol) hay varios proveedores y se puede cambiar sin tocar el resto. Si uno falla o se queda sin cuota, el sistema salta al siguiente automáticamente.

Tabla de servicios externos

Servicio	Para qué	Clave / Env	¿Gratis?
Cerebras	LLM principal (narraciones, textos)	<code>CEREBRAS_API_KEY</code> (×5)	Gratis (límite)
Groq	LLM de respaldo + editor de estilo	<code>GROQ_API_KEY</code>	Gratis
Gemini	LLM alternativo de Google (opcional)	<code>GEMINI_API_KEY</code>	Gratis (límite)
Ollama	LLM local sin internet (opcional)	<code>OLLAMA_HOST</code>	Local
Together.ai	Imágenes de fondo (FLUX.1-schnell)	<code>TOGETHER_API_KEY</code> (×2)	Gratis (límite)
Fal.ai	Imágenes (respaldo)	<code>FAL_API_KEY</code>	Gratis (límite)
Edge-TTS	Voz del narrador (por defecto)	— sin clave	Gratis
ElevenLabs	Voz premium	<code>ELEVEN_LABS*</code>	Gratis (límite) / pago
ESPN	Datos de fútbol (goles, minutos)	<code>ESPN_LEAGUE_SLUG</code>	Gratis
arXiv + bge-small + Chroma	Ciencia (RAG)	— local	Local
Finnhub + yfinance	Finanzas (precio + titulares)	<code>FINNHUB_API_KEY</code>	Gratis (límite)
YouTube Data API v3	Subir vídeos	OAuth por perfil	Gratis (cuota)
LangSmith	Registro/depuración de llamadas IA	<code>LANGSMITH_API_KEY</code>	Gratis (límite)

🔧 **Detalle técnico.** El orden de preferencia para resolver cualquier ajuste es: `profile.json` → `.env` del perfil → `.env` global → valor por defecto del código. La rotación de claves (p. ej. las 5 de Cerebras) se dispara con el error HTTP 429 (cuota agotada): se prueba la siguiente clave hasta que una funciona.

Parte 3 · Flujo de Vídeos (Partidos → .mp4)

Es el flujo principal y el más complejo. Convierte un partido terminado en un vídeo narrado con marcador animado y subtítulos, listo para YouTube.

- **Entra:** un partido (equipos + marcador).

- **Sale:** un archivo `match_<id>.mp4`.
- **Orquesta:** `pipeline/runner.py · run_match()`.
- **Se lanza con:** `POST /api/profiles/{id}/generate` (o desde la página Partidos).

A continuación, los 9 pasos. Cada uno recibe algo, hace una cosa y produce algo para el siguiente.

Paso 1 · Enriquecer datos (goleadores)

A veces solo conocemos el marcador (p. ej. 2-0) pero no quién marcó. Este paso llama a **ESPN** para rellenar: nombres de goleadores, minutos, tarjetas y una breve descripción de cada gol.

- **Entra:** partido con marcador. **Sale:** partido con goles detallados.
- **Archivo:** `pipeline/data_sources/espn_enrich.py`.

Por qué importa. Sin esto la narración sería vaga (“hubo dos goles”). Con esto puede decir: *“gol de Pérez al minuto 70 tras un contragolpe”*.

Paso 2 · Escribir la narración

Se ordenan los hechos por orden cronológico y se le pasan a un **LLM** con instrucciones de actuar como relator legendario.

- **Entra:** hechos del partido. **API:** Cerebras/Groq. **Sale:** guion hablado.
- **Archivo:** `pipeline/narrator.py`.

 **Detalle técnico.** El prompt pide 90-150 palabras, apto para todo público, con gramática española correcta. Construye un “bloque de hechos” (`_facts_block`) para que el modelo no tenga que adivinar nada.

Paso 3 · Guardrail (verificar que no inventa)

Dos comprobaciones encadenadas:

1. **Automática:** una regla comprueba que el marcador del texto coincide con el real.
2. **Juez IA:** un segundo LLM verifica que todo esté respaldado por los hechos, en el idioma correcto y con buen tono. Devuelve un veredicto en formato estructurado.

Si algo falla, **se reescribe una vez** siendo más estricto.

- **Archivo:** `agents/guardrail.py`.

Ejemplo. Si la narración dijera “gol de Messi” pero Messi no aparece en los datos, el juez lo marca como no fundamentado y se regenera.

Paso 4 · Pulir el texto

Un LLM editor reescribe frases que suenan raras y corrige fallos típicos del español (p. ej. “la penalty” → “el penalti”). Después **se vuelve a verificar** que no haya cambiado ningún dato; si el editor alteró un nombre por error, se descarta su versión.

- **Archivo:** `pipeline/text_polish.py`.

Paso 5 · Metadata de YouTube

El LLM crea un **título** atractivo (con el marcador, máx. 90 caracteres) y una **descripción**. Las **etiquetas** se construyen automáticamente (competición, equipos, países, goleadores, estadio).

- **Archivo:** `pipeline/narrator.py`.

Paso 6 · Imágenes de ambiente (opcional)

Si está activado, se genera un fondo de grada con los colores del equipo ganador usando **FLUX** (un modelo que crea imágenes a partir de texto).

- **Entra:** nombre del equipo. **API:** FLUX.1-schnell (Together/Fal). **Sale:** PNG.
- **Archivo:** `pipeline/media_provider.py`.

Nota. Por defecto el vídeo usa gráficos animados propios (marcador, escudos), así que este paso suele omitirse salvo que actives “flux” en las fuentes de medios.

Paso 7 · Voz y subtítulos (TTS)

El motor de voz lee el guion y produce el audio. Antes:

- Limpia los “GOOOL” estirados a “gol” para que la voz no se trabe.
- Sube la energía cuando detecta gritos (signos `!!` y mayúsculas).

Devuelve también el **momento exacto de cada palabra**, que servirá para los subtítulos sincronizados.

- **API:** Edge-TTS (Jorge) o ElevenLabs. **Sale:** `.mp3` + tiempos por palabra.
- **Archivo:** `pipeline/voice_generator.py`.

Paso 8 · Montar el vídeo

Se combinan todas las piezas:

- Gráficos animados (marcador, escudos, cronología de goles y tarjetas).
- Subtítulos estilo **karaoke** (una palabra grande cada vez, sincronizada con la voz).
- La pista de audio de la narración.
- **Herramienta:** MoviePy (códecs H.264/AAC). **Sale:** `match_<id>.mp4`.

- **Formato:** reel vertical 1080×1920 o YouTube horizontal 1920×1080.
- **Archivo:** `pipeline/video_assembler.py`.

Paso 9 · Subir a YouTube (opcional)

Si lo pides o tienes la subida automática activada, se publica el vídeo en tu canal con el título y descripción generados, usando el permiso **OAuth** ya guardado.

- **API:** YouTube Data API v3. **Archivo:** `pipeline/publishers.py`.

Seguro. En modo práctica todo se sube como **privado**. Si la subida falla, el vídeo no se pierde: queda en la Biblioteca para subirlo a mano.

Variante · Resumen del día (digest)

En lugar de un partido, junta **todos los partidos de un día** en un solo vídeo (narración corta por partido, concatenadas). El reel se limita a 6 partidos.

- **Archivo:** `pipeline/digest.py`.

Parte 4 · Flujo de Crear (texto multiplataforma)

Escribe publicaciones adaptadas a cada red social sobre cualquier tema.

- **Entra:** tema + público + plataformas (blog, X, Instagram, LinkedIn).
- **Sale:** un texto a medida para cada plataforma.
- **Orquesta:** `pipeline/content_generator.py · generate_freeform()`.
- **Se lanza con:** `POST /api/profiles/{id}/content/freeform`.

Cómo funciona

Cada red tiene su “molde”. Para cada plataforma elegida, se le da al LLM el molde adecuado y genera el texto.

Plataforma	Formato
Blog	250-400 palabras, con título (H1)
X (Twitter)	máx. 280 caracteres, 2-3 hashtags
Instagram	~150 palabras, 5-8 hashtags
LinkedIn	~120 palabras, tono más profesional

Antes del prompt se añade la **personalidad de tu marca** (el `system_preamble` del perfil) y una instrucción explícita de **no inventar** hechos.

Ejemplo. Tema “la táctica del fuera de juego”, público “general”, plataformas “X + LinkedIn” → genera un tweet corto con hashtags y, aparte, un post de LinkedIn más reposado, ambos sobre el mismo tema pero con tono y longitud distintos.

🔧 **Detalle técnico.** Existe además un modo “partido” (`generate_all`) que ata el texto a los hechos reales del encuentro (“nunca inventes goleadores”).

Parte 5 · Laboratorio · Ciencia (RAG sobre arXiv)

Explica un tema científico apoyándose en **papers reales** de arXiv (el gran repositorio de artículos científicos).

- **Entra:** un tema (p. ej. “biomecánica del sprint”).
- **Sale:** una explicación divulgativa fundamentada en papers reales.
- **Orquesta:** `pipeline/tools/arxiv_rag.py · explain()`.
- **Se lanza con:** `POST /api/science/explain`.

Paso 1 · Descargar e indexar papers

La primera vez que consultas un tema:

1. Se bajan hasta **8 artículos** de arXiv sobre el tema.
2. Se cortan en **trozos** de ~1000 caracteres (con 150 de solape, para no partir ideas a la mitad).
3. Cada trozo se convierte en **embeddings** (números) con `bge-small`.
4. Se guardan en **Chroma**, una base de datos que busca por significado.

Esto se hace una sola vez por tema y queda guardado (cacheado).

Analogía. Es como subrayar y fichar todos los papers para luego encontrar al instante el párrafo exacto que responde a una pregunta, sin releerlo todo.

Paso 2 · Recuperar lo relevante

Tu pregunta también se convierte en embeddings y se buscan en Chroma los **4 trozos** cuyo significado más se le parece. Si activas el **grafo** (Graph RAG), además se extraen relaciones entre conceptos (“A causa B”) como contexto extra.

- **Archivo:** `pipeline/tools/graph_rag.py`.

Paso 3 · Redactar la explicación

Se le pasan al LLM **solo** los trozos recuperados y se le pide explicar el tema usando únicamente esa información. Como se apoya en papers reales, no inventa.

🔑 **Detalle técnico.** `bge-small` corre en local (sin clave), Chroma también. Por eso este flujo está etiquetado como “Local”: no envía nada a internet salvo la descarga inicial de arXiv.

Parte 6 · Laboratorio · Finanzas

Resumen de bolsa de una empresa: precio actual y titulares recientes.

- **Entra:** un *ticker* (p. ej. AAPL, NVDA).
- **Sale:** precio, variación del día y titulares.
- **Orquesta:** `pipeline/tools/finance.py · market_summary()`.
- **Se lanza con:** `POST /api/finance/news`.

Cómo funciona

1. **Precio:** se pide a **Finnhub**. Si no responde (o no hay clave), se usa **yfinance** (Yahoo Finanzas, sin clave) como respaldo, calculando la variación comparando con el cierre anterior.
2. **Titulares:** se piden a Finnhub las 5 noticias más recientes. Si no hay clave, simplemente no se muestran (el precio sigue funcionando).

Ejemplo de salida. `NVDA: $203.26 (-1.53% hoy)` seguido de 5 titulares recientes de la empresa.

Parte 7 · Laboratorio · Agentes (LangGraph)

Un “jefe” (supervisor) decide qué experto responde tu petición y delega en él.

- **Entra:** una petición en lenguaje libre.
- **Sale:** la respuesta del agente experto más adecuado.
- **Orquesta:** `agents/graph.py · route_request()`.
- **Se lanza con:** `POST /api/agents/route`.

Cómo funciona

1. **El router** (un LLM con temperatura 0, es decir, respuesta determinista) lee tu petición y elige **un** experto.
2. **Los 4 expertos:**
 - **sports** — contenido factual de fútbol.
 - **social** — copys para redes.
 - **science** — puede usar la herramienta de arXiv para fundamentarse.

- **finance** — usa la herramienta de Finnhub para dar cifras reales.
3. El experto responde (llamando a su herramienta si hace falta) y el supervisor devuelve el resultado.

Ejemplo. Si escribes “resume las noticias de Apple”, el router lo manda al agente *finance*, que llama a Finnhub y devuelve el resumen. Si escribes “escribe un tweet de la final”, va al agente *social*.

🔧 **Detalle técnico.** Cada agente es un *react agent* (`create_react_agent`): puede razonar y decidir por sí mismo cuándo invocar su herramienta antes de responder. Todos corren sobre el `LLM_PROVIDER` configurado.

Parte 8 · La web (frontend) y los Ajustes

La interfaz tiene estas páginas:

- **Dashboard** — resumen y accesos rápidos.
- **Partidos** — elegir un partido y generar su vídeo.
- **Calendario** — el calendario del Mundial 2026.
- **Crear** — texto multiplataforma.
- **Laboratorio IA** — ciencia, finanzas y agentes.
- **Biblioteca** — los vídeos generados; publicar/programar subidas.
- **Arquitectura** — documentación viva del sistema (modo Fácil/Técnico, desplegable).
- **Ajustes** — dos pestañas:
 - **Aplicación:** idioma (español/inglés) y tema (claro/oscuro/automático).
 - **Perfil de fútbol:** voz, competición, proveedor LLM, proveedor de imagen y automatización de YouTube.

🔧 **Detalle técnico.** El idioma y el tema se guardan en el navegador (`localStorage`). El tema “automático” sigue al sistema operativo. La página de Arquitectura se alimenta del endpoint `GET /api/architecture` , que describe el sistema desde el propio código.

Apéndice · Mapa rápido de endpoints

Área	Endpoint	Qué hace
Config	GET /api/config/global	Opciones disponibles (voces, idiomas...)
Perfiles	GET/POST/PATCH /api/profiles/{id}	Listar, crear, editar perfiles
Partidos	GET /api/profiles/{id}/matches	Partidos (por día o últimos)
Vídeo	POST /api/profiles/{id}/generate	Generar el vídeo de un partido
Resumen día	POST /api/profiles/{id}/digest	Vídeo con todos los partidos del día
Estado	GET /api/profiles/{id}/status	Progreso de la generación en vivo
Biblioteca	GET /api/profiles/{id}/content	Contenido generado
Publicar	POST /api/profiles/{id}/publish	Subir un vídeo a YouTube
Crear	POST /api/profiles/{id}/content/freeform	Texto multiplataforma
Ciencia	POST /api/science/explain	Explicación científica (RAG)
Finanzas	POST /api/finance/news	Resumen de mercado
Agentes	POST /api/agents/route	Enrutado multiagente
Arquitectura	GET /api/architecture	Mapa del sistema

Documento generado para SynapseQuill · FIFA World Cup 2026.